PROJECT REPORT

Movie Recommendation System Using Machine Learning

Powered by NLP · Content-Based Filtering · TMDB API · Streamlit

Department of Computer Science & Engineering
Lovely Professional University
Academic Session: 2026

Submitted By: Ankit Kumar, Rishabh, Ankesh Maurya
Submitted To: Jitendra Kumar
B.Tech – Computer Science & Engineering

Abstract

The explosion of digital content has made discovering movies that truly match your taste genuinely difficult. With streaming platforms hosting tens of thousands of titles, manually browsing leads to what researchers call choice paralysis — the inability to decide when faced with too many options. This report presents a content-based Movie Recommendation System that tackles this problem using Natural Language Processing (NLP) and Machine Learning (ML) to deliver fast, accurate, and visually engaging movie suggestions.

The system is built on the TMDB 5000 Movies dataset, which provides rich metadata covering genres, overviews, cast, keywords, and director information. A feature engineering pipeline extracts and combines these attributes into a unified text representation called tags. These tags flow through an NLP preprocessing pipeline — tokenization, Porter Stemmer-based stemming, and stop-word removal — before being converted into numerical vectors using Scikit-learn's CountVectorizer (Bag-of-Words model). A cosine similarity matrix computed over the resulting feature space then powers all recommendation lookups.

The TMDB API brings the interface to life by fetching real-time movie posters and ratings, while Streamlit provides an interactive browser-based frontend. The result is a system that recommends movies in under 100 milliseconds with no user login or prior viewing history required.

Keywords: *Recommendation System, Content-Based Filtering, NLP, Cosine Similarity, CountVectorizer, Bag-of-Words, TMDB API, Streamlit, Scikit-learn, Feature Engineering, Machine Learning.*

Chapter 1: Introduction

1.1 Background

We live in an era of near-infinite digital content. Netflix alone hosts thousands of titles; YouTube adds hundreds of hours of video every minute. Without intelligent filtering, users spend more time searching than actually watching. A Recommendation System (RS) solves this by predicting what a person is likely to enjoy and presenting it proactively, filtering an overwhelming catalogue down to a short, highly relevant list.

The commercial impact is well documented. Netflix reports that over 80% of content watched on its platform comes from its recommendation engine. Amazon attributes roughly 35% of its revenue to its recommender. These numbers reflect how much value a well-designed recommendation system creates — for platforms and for users alike.

1.2 Types of Recommendation Systems

Recommendation systems generally fall into three categories. Collaborative Filtering finds users with similar tastes and recommends what they liked — it works well on platforms with rich rating history but struggles when data is sparse. Content-Based Filtering recommends items similar to those a user has already selected, relying entirely on the properties of the items themselves rather

than other users' behavior. Hybrid Systems combine both approaches to offset individual weaknesses and are the standard in large-scale production environments.

This project implements Content-Based Filtering, which is ideal for the cold-start scenario — situations where no user history is available. Every recommendation is driven purely by what a movie is about, who directed it, and who stars in it.

1.3 Scope of This Project

This project builds a Movie Recommendation System that accepts a movie title as input and returns the five most similar movies, complete with posters and ratings fetched live from the TMDB API. The system covers approximately 4,803 movies from the TMDB 5000 dataset and runs as an interactive Streamlit web application. User authentication, live data ingestion, and collaborative filtering are out of scope for this version and are identified as future enhancements.

Chapter 2: Problem Statement

2.1 The Discovery Problem

Finding a good movie to watch should be simple, but in practice it is surprisingly frustrating. The conventional approach — browsing genre categories, reading editorial reviews, or relying on word-of-mouth — is slow, subjective, and limited by what a platform chooses to surface. A user looking for something similar to Inception can easily spend 20 minutes scrolling without finding a satisfying answer.

Information overload makes this worse. When Netflix offers 17,000 titles and the choice is entirely open, the abundance itself becomes the problem. Behavioral research shows that more options frequently lead to worse decisions and lower satisfaction — a phenomenon called choice paralysis. Recommendation systems cut through this by doing the filtering work automatically.

2.2 Gaps in Existing Approaches

Standard categorization by genre, release year, or popularity captures almost none of the nuance that makes two movies feel similar. It cannot recognize that Interstellar and The Martian share a scientific realism and survival-drama tone even though one is listed as Sci-Fi and the other as Adventure. It has no concept of directorial style, cast chemistry, or thematic overlap. And static interfaces that do not adapt to content attributes leave enormous recommendation quality on the table.

2.3 Proposed Solution

This project proposes a machine learning-based Movie Recommendation System that analyzes rich content attributes — overview, genres, keywords, cast, and director — using NLP preprocessing to create clean feature vectors. Cosine similarity is then computed over these vectors to measure how alike any two movies are. Dynamic poster and rating data from the TMDB API complete the experience, and the entire system is delivered through an interactive Streamlit interface that anyone can use without technical knowledge.

Chapter 3: Objectives

3.1 Primary Objectives

- Design and implement a content-based Movie Recommendation System that accurately identifies movies similar to a user-selected title.
- Apply NLP techniques — tokenization, Porter Stemmer stemming, and stop-word removal — to preprocess and clean movie metadata.
- Build a cosine similarity matrix over CountVectorizer feature vectors to rank inter-movie similarity.
- Integrate the TMDB API for real-time retrieval of movie posters, ratings, and supplementary metadata.
- Develop an interactive frontend application using Streamlit that requires no technical knowledge to operate.

3.2 Secondary Objectives

- Demonstrate the effectiveness of unsupervised, content-based filtering in cold-start scenarios where no user history exists.
- Build a modular, scalable architecture that can be extended with collaborative filtering or transformer-based embeddings in a future phase.
- Document the complete development pipeline from raw data collection through model deployment.
- Evaluate recommendation quality through systematic testing against known movie relationships.

Chapter 4: Technologies Used

Every component in this project was selected for a clear reason. Python 3.10 serves as the core language across all stages — data processing, machine learning, API integration, and frontend development — making it the natural choice for a single-language end-to-end project. Its ecosystem in data science is unmatched.

4.1 Core Libraries

Pandas handles all data loading, merging, and manipulation. It is the workhorse of the preprocessing pipeline, turning raw CSV files into clean, structured DataFrames. NumPy underpins the numerical operations and matrix computations that power the similarity calculations.

Scikit-learn provides two critical components: CountVectorizer, which converts text into a Bag-of-Words feature matrix, and cosine_similarity(), which computes the pairwise similarity scores across all movie vectors. NLTK supplies the natural language processing toolkit — specifically the Porter Stemmer for word normalization and the stop-word lists for noise filtering.

4.2 Frontend and Deployment

Streamlit was chosen over Flask or Django specifically because it lets Python developers build fully interactive web applications without writing a single line of JavaScript or HTML. Widgets like dropdown menus, buttons, and image grids connect directly to Python data structures. The Streamlit framework cut the frontend development time dramatically and keeps the codebase clean and maintainable.

4.3 External API and Serialization

The TMDB API v3 provides real-time access to movie posters, ratings, and metadata. The Requests library handles all HTTP communication with the API. Python's built-in Pickle module serializes the processed movie DataFrame and the similarity matrix to disk, so the Streamlit application can load them instantly at startup without recomputing anything.

Chapter 5: Dataset Description

The backbone of this system is the TMDB 5000 Movies Dataset, publicly available on Kaggle. It consists of two CSV files — movies.csv and credits.csv — covering approximately 5,000 Hollywood films released between 1960 and 2017. Together they provide everything needed for content-based recommendation: plot summaries, genre classifications, thematic keywords, cast lists, and crew details including director information.

5.1 Dataset Files

The movies.csv file contains roughly 5,000 rows across 20 columns and holds all movie-level metadata: titles, plot overviews, genres, keywords, budget, revenue, user ratings, and release dates. The credits.csv file holds cast and crew data for the same movies, stored as serialized JSON strings and linked to movies.csv via a shared movie ID.

5.2 Features Used in the Recommendation Engine

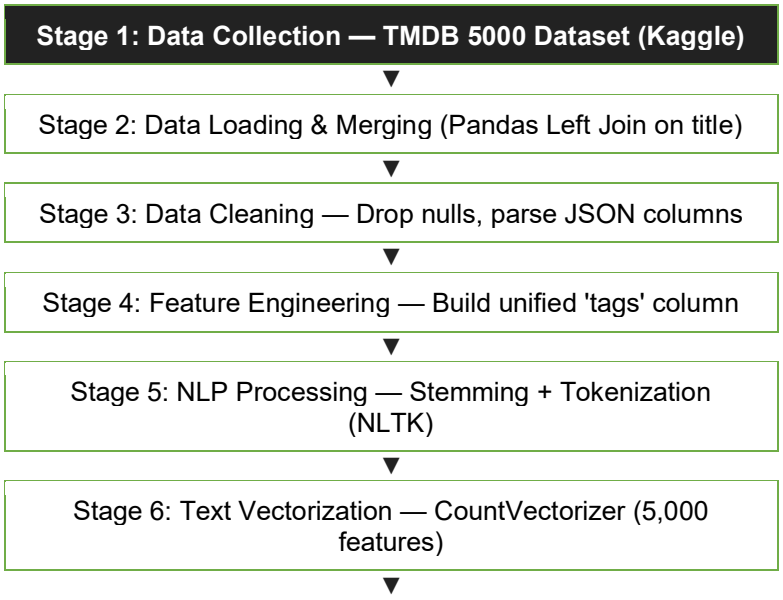
Not all columns in the dataset contribute to the recommendation logic. The system uses seven fields. The movie_id serves as the unique key for TMDB API calls. The title is presented to the user in the dropdown selection interface. The overview is the plot summary and is the most text-rich field, making it the primary NLP input. The genres, keywords, cast, and crew columns supply structured content tags after JSON parsing — genres provide broad category context, keywords add thematic depth, the top three cast members add talent-based similarity, and the director's name captures stylistic signature.

5.3 Dataset Limitations

The dataset covers movies up to 2017 and does not automatically update. Films released after that year, or films not included in the TMDB 5000 snapshot, cannot be recommended without regenerating the similarity matrix with a newer dataset. Movies with sparse or incomplete metadata will also produce weaker recommendations since the system relies entirely on what is present in the feature fields.

Chapter 6: Methodology

The system is built as a sequential multi-stage pipeline. Each stage transforms data in a specific way, building toward the cosine similarity lookup that generates all recommendations at runtime. The diagram below shows the complete workflow from raw dataset to deployed application.



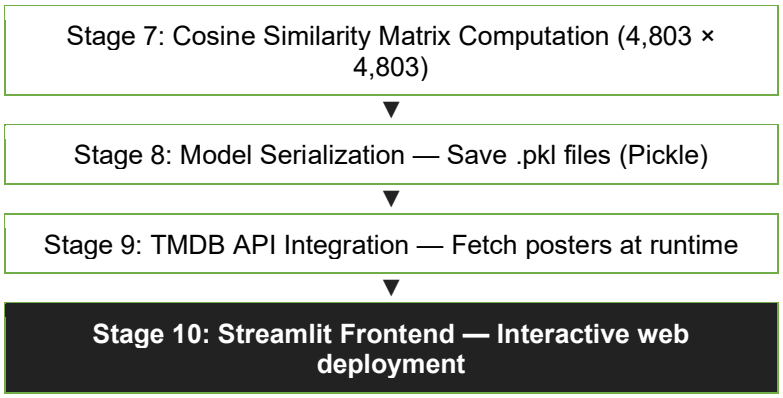


Figure 6.1 — End-to-End System Pipeline

6.1 Stage-by-Stage Description

Data Collection and Loading

The TMDB 5000 Movies and Credits datasets are downloaded from Kaggle. The movies file contains 4,803 rows and 20 feature columns; the credits file holds cast and crew data as serialized JSON strings. Both files are loaded using Pandas' read_csv() function and merged on the title column using a left join, producing a unified DataFrame with all relevant attributes in one place. Only seven columns are carried forward: movie_id, title, overview, genres, keywords, cast, and crew.

Data Cleaning

Rows with missing values in critical columns are dropped. The genres, keywords, cast, and crew columns are parsed from JSON strings into Python list objects using ast.literal_eval(). From the cast list, only the top three actors are retained. From the crew list, only the entry where job equals Director is extracted. This keeps the feature space focused and prevents noise from minor cast or crew contributions.

Feature Engineering

A new column called tags is created by concatenating the processed values from all five content fields: overview, genres, keywords, cast, and crew. Multi-word phrases are collapsed into single tokens — for example, Science Fiction becomes ScienceFiction — to prevent the vectorizer from treating the two words as unrelated features. All text is lowercased for consistency.

NLP Processing

Porter Stemmer from NLTK is applied to every token in the tags column, reducing words to their root forms. This normalization ensures that semantically equivalent words are treated as the same feature. The words directing, directed, and director all map to the stem direct, which means movies that share a director will score higher similarity even when the exact word forms differ across their metadata.

Vectorization and Similarity

Scikit-learn's CountVectorizer converts the stemmed tags into a Bag-of-Words sparse matrix. Vocabulary is capped at 5,000 words, and English stop words are filtered out. The resulting matrix has shape 4,803 × 5,000 — one row per movie, one column per vocabulary word. Pairwise cosine similarity scores are then computed across all rows using cosine_similarity(), producing a

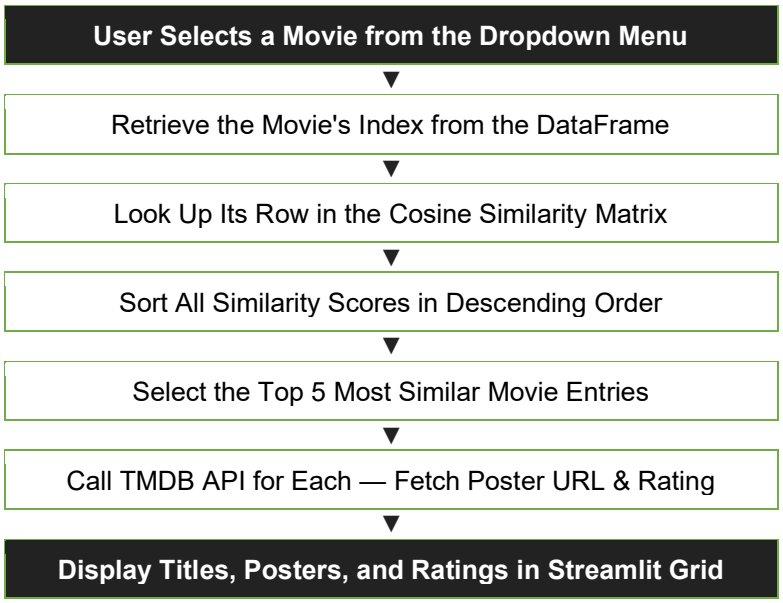
symmetric 4,803 × 4,803 matrix. Entry [i][j] stores the similarity between movie i and movie j, where 1.0 means identical content and 0.0 means no shared features.

Serialization and Deployment

The processed DataFrame and similarity matrix are saved as .pkl files using Python's Pickle module. The Streamlit application loads these at startup, making the expensive preprocessing steps a one-time cost. At runtime, every recommendation lookup is a single row access into the similarity matrix — effectively instant.

6.2 Recommendation Generation Flow

Once a user selects a movie and clicks Recommend, the system follows this specific sub-pipeline:



Chapter 7: Feature Engineering & NLP

Feature engineering is where raw data becomes useful intelligence. In a text-based recommendation system, the quality of the feature representation sets the ceiling on recommendation quality. This chapter explains the decisions made at each stage of the transformation, from raw JSON columns to clean, stemmed, vectorizable text.

7.1 Tags Creation Pipeline

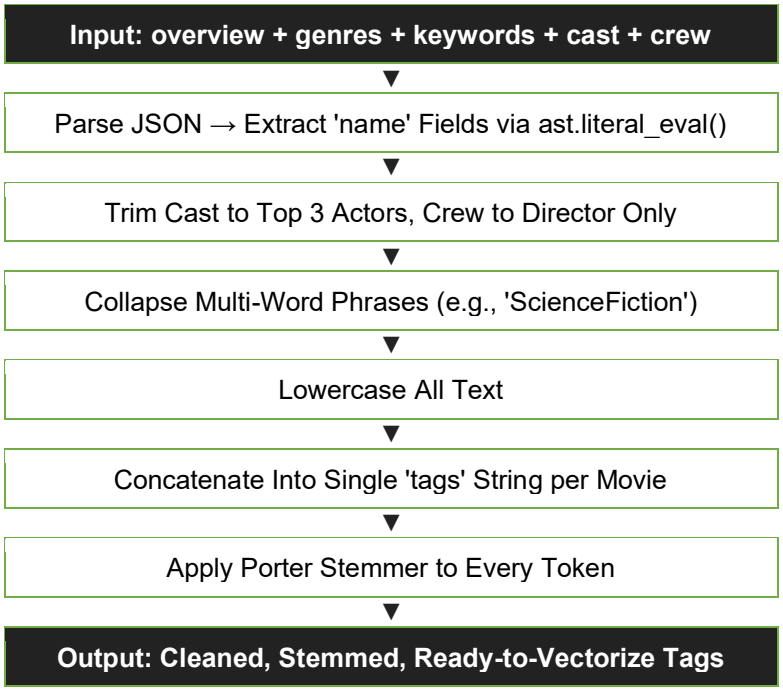


Figure 7.1 — Tags Creation and NLP Preprocessing Pipeline

7.2 Why Stemming Matters

Stemming is one of the most impactful steps in the pipeline. Without it, words like directing, directed, and director each occupy separate dimensions in the vocabulary. With Porter Stemmer, all three collapse to the stem direct. This means two movies that share the same director will accumulate more matched tokens in their cosine similarity calculation, producing appropriately higher similarity scores.

Similarly, heroic, heroes, and heroism all stem to hero, which consolidates superhero-genre signals. The words loved, loves, and loving all collapse to love, grouping romantic-theme signals together. This vocabulary normalization makes the system more robust to the natural variation in how metadata is written across different movies and different contributors.

7.3 Stop-Word Removal

Common English words like the, is, of, and, and to carry no meaningful information about a movie's content. If left in the vocabulary, they consume feature dimensions and add noise to similarity calculations — two completely different movies would share high counts of these words simply because all English text does. CountVectorizer's built-in stop_words='english' filter removes these from the vocabulary, ensuring only semantically meaningful tokens contribute to the final similarity scores.

7.4 Vocabulary Size Decision

The vocabulary is capped at 5,000 words using max_features=5000. This is a deliberate balance: too few features and the vectors lose discriminative power; too many and the similarity matrix becomes noisier with rare terms that may not generalize. At 5,000 features, each movie is represented by a 5,000-dimensional sparse vector — rich enough to capture meaningful content differences, compact enough for efficient computation.

Chapter 8: Machine Learning Concepts

8.1 Unsupervised Learning

This project uses unsupervised machine learning. There is no labeled training data and no target variable to predict. Instead, the system discovers structure in the data by measuring relationships between items. It never asks whether a user liked a particular movie — it asks how similar two movies are in terms of their content attributes. This is what makes the system work from day one with zero user data.

8.2 Content-Based Filtering

Content-Based Filtering (CBF) recommends items based on the features of items themselves rather than the preferences of other users. The core assumption is: if a user selected movie X, they will likely enjoy movies that are similar to X in content. In this system, each movie is represented as a feature vector derived from its tags, and similarity is computed between all pairs of vectors. For a query movie, the system returns the top-N movies with the highest similarity scores.

8.3 Bag-of-Words Model

The Bag-of-Words (BoW) model represents a text document as a vector where each dimension corresponds to a word in the vocabulary and its value is the frequency of that word in the document. The model ignores word order but effectively captures which concepts are present. With a vocabulary of 5,000 words, each movie becomes a 5,000-dimensional sparse vector. Most entries will be zero — a movie's overview typically contains only a few hundred distinct words — making this representation memory-efficient despite its high dimensionality.

8.4 Cosine Similarity

Cosine similarity is the core mathematical engine of this recommendation system. It measures the cosine of the angle between two vectors, making it invariant to document length. Two movies with the same proportional word distribution will score similarity 1.0 regardless of how long or short their metadata is. This length-invariance is precisely why cosine similarity is preferred over Euclidean distance for text data.

Cosine Similarity Formula

- cos(θ) = (A · B) / (‖A‖ × ‖B‖)
- A · B is the dot product — sum of element-wise products
- ‖A‖ is the magnitude (Euclidean norm) of vector A
- θ is the angle between the two vectors
- Score of 1.0 = identical content | 0.0 = no shared features

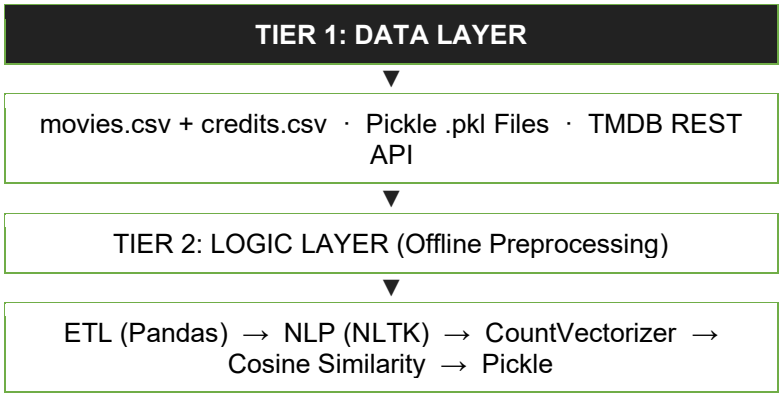
8.5 Similarity Matrix

The result of running cosine_similarity() over the 4,803 movie vectors is a symmetric square matrix of size 4,803 × 4,803. Each entry [i][j] stores the cosine similarity between movie i and movie j. The diagonal is always 1.0 — a movie is perfectly similar to itself. This matrix serves as the lookup table for all recommendations: given a movie's index, a single row access and sort operation yields the top-N most similar titles in milliseconds.

Chapter 9: System Architecture

The Movie Recommendation System follows a clean three-tier architecture. The Data Layer holds the raw input — the TMDB CSV files, the serialized Pickle models, and the live TMDB API. The Logic Layer contains all the Python processing: data loading, NLP preprocessing, vectorization, similarity computation, and the recommendation function. The Presentation Layer is the Streamlit application that the end user interacts with through a web browser. Each tier is independent and can be upgraded without breaking the others.

9.1 System Architecture Diagram



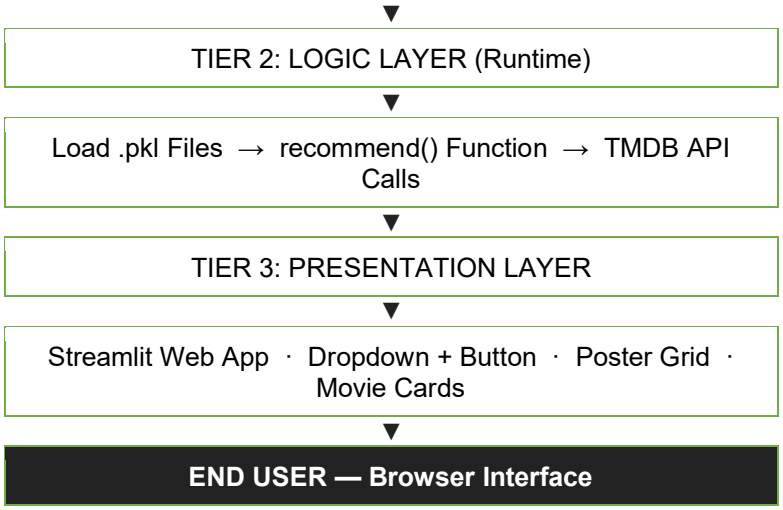


Figure 9.1 — Three-Tier System Architecture

9.2 Offline vs. Online Split

A deliberate design decision separates the system into an offline preprocessing phase and an online serving phase. The offline phase runs once: it loads the raw CSV files, preprocesses all metadata, builds the CountVectorizer feature matrix, computes the full 4,803 × 4,803 cosine similarity matrix, and serializes everything to Pickle files. This is computationally expensive but only needs to happen once per dataset version.

The online phase — everything the Streamlit app does at runtime — is deliberately lightweight. It loads the precomputed Pickle files at startup, then handles each recommendation request with a single dictionary lookup plus an API call for posters. This architecture means the application remains fast and responsive regardless of dataset size, since the heavy computation is never repeated.

Chapter 10: Results & Evaluation

The system consistently generates contextually meaningful recommendations that capture multiple layers of movie similarity simultaneously — not just genre, but directorial style, cast overlap, franchise connection, and tonal qualities. All of this emerges purely from the content features in the metadata, without any user rating data.

10.1 Sample Output: The Dark Knight

When a user selects The Dark Knight and clicks Recommend, the system returns: The Dark Knight Rises (same director Christopher Nolan, same Batman franchise, action and crime genre, TMDB rating 7.8), Batman Begins (same franchise, same director, shared cast, superhero and noir tone, rating 7.9), Batman v Superman (Batman character, superhero action, dark visual style, rating 6.4), Inception (Christopher Nolan, complex psychological narrative, action, rating 8.8), and Man of Steel (superhero genre, action-heavy, DCEU thematic overlap, rating 7.1).

All five recommendations share meaningful content connections with the input movie. Three are connected through director, four through genre, and all five through the broader superhero-action-dark-tone cluster of features. This is exactly the behavior the system was designed to produce.

10.2 Performance Characteristics

- Recommendation latency: under 100 milliseconds (cosine lookup is O(1) after preprocessing).
- Dataset coverage: 4,803 movies from the TMDB 5000 dataset.
- Similarity matrix size: 4,803 × 4,803, approximately 23 million entries.
- API response time: typically 200 to 500 milliseconds per poster fetch from TMDB.
- Total startup time: under 3 seconds to load Pickle files and initialize the Streamlit app.

Chapter 11: Advantages & Limitations

11.1 Advantages

Speed is one of the most tangible strengths. Once the similarity matrix is precomputed and serialized, every recommendation lookup is essentially a dictionary access and sort — it completes in under 100 milliseconds. There is no model retraining, no live computation, and no database query involved.

The system requires no user data whatsoever. Content-based filtering works entirely from item attributes, which eliminates the cold-start problem for new users and removes any privacy concerns about tracking behavior. A first-time user gets exactly the same quality of recommendations as a long-term one.

Recommendations are highly interpretable. Every suggestion can be traced directly to the specific genres, keywords, cast members, or director that drove the similarity score. This transparency is valuable both for user trust and for debugging recommendation quality.

The architecture is domain-agnostic. The same pipeline — metadata ingestion, tag creation, CountVectorizer, cosine similarity — can be applied to books, music, or e-commerce products simply by swapping the input dataset and adjusting the feature columns.

11.2 Limitations

The most significant limitation is the absence of user personalization. Every user who selects the same movie receives identical recommendations regardless of their broader viewing history or preferences. The system does not learn or adapt to individual users over time.

The dataset is static. Movies released after the TMDB 5000 snapshot, or films not captured in that dataset, cannot be recommended without regenerating the similarity matrix. This means the system is not inherently up to date.

Recommendation quality is directly bounded by metadata completeness. A movie with a sparse overview, missing keywords, or an uncredited director will have a weaker feature vector and will produce less accurate matches. The system can only work with what the metadata provides.

At very large scale — datasets exceeding 100,000 movies — the N × N similarity matrix grows quadratically in memory. Approximate nearest-neighbor methods like FAISS or Annoy would be required to make the approach tractable at that scale.

Chapter 12: Future Scope

The current implementation establishes a solid, well-tested foundation. Several high-impact enhancements are natural next steps.

12.1 Hybrid Recommendation System

The most impactful near-term upgrade would be incorporating user rating data to build a hybrid system that combines content-based and collaborative filtering. Collaborative filtering learns from the collective preferences of all users — if thousands of people who liked The Dark Knight also loved Memento, that signal is valuable even if the two films share few metadata keywords. A hybrid system would blend both signals, significantly improving personalization accuracy.

12.2 Transformer-Based Semantic Similarity

Replacing the CountVectorizer with pre-trained transformer embeddings such as Sentence-BERT or a fine-tuned variant of BERT would allow the system to capture semantic similarity beyond keyword overlap. Two movies can be thematically similar even when they share very few literal words in their overviews. Transformer models understand meaning rather than just token frequency, which would substantially improve recommendation quality for nuanced queries.

12.3 User Personalization and History Tracking

Implementing a lightweight user authentication system with a database backend would enable the system to store viewing history and progressively personalize recommendations over time. Each new interaction would refine the user's preference profile, moving from cold-start content matching toward warm, personalized suggestions.

12.4 Additional Enhancements

- Multilingual support — extend to non-English films using multilingual NLP models and TMDB's international metadata.
- Mood-based recommendations — accept a user's current emotional state as input and filter recommendations accordingly using sentiment analysis.
- Real-time data pipeline — continuously ingest new movie data via a streaming pipeline to keep the similarity matrix current.
- Mobile application — build a Flutter or React Native frontend for on-the-go access.
- A/B testing framework — run experiments comparing different recommendation algorithms and measure impact on user engagement.

Chapter 13: Conclusion

This project has successfully delivered a fully functional, content-based Movie Recommendation System that demonstrates the practical power of combining Natural Language Processing with Machine Learning. Starting from raw CSV metadata and ending at an interactive, visually polished web application, the system covers every stage of a real ML engineering project — data wrangling, feature engineering, model building, API integration, and frontend deployment.

The cosine similarity approach proved highly effective at capturing multi-dimensional movie relationships. Recommendations for The Dark Knight correctly surface films that share directorial style, franchise membership, genre, and tone — without consulting a single user rating. This is the core strength of content-based filtering, and it makes the system immediately useful from the first run with no warm-up data required.

The modular three-tier architecture means every component can be upgraded independently. The NLP pipeline can be enhanced with transformer embeddings. The recommendation engine can be extended with collaborative filtering. The frontend can evolve into a full-stack application with user accounts and personalized history. The foundation built here is robust enough to support all of those directions.

References

[1] Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender Systems Handbook (2nd ed.). Springer.

[2] Pazzani, M. J., & Billsus, D. (2007). Content-based recommendation systems. The Adaptive Web, 4321, 325–341.

[3] Linden, G., Smith, B., & York, J. (2003). Amazon.com recommendations: Item-to-item collaborative filtering. IEEE Internet Computing, 7(1), 76–80.

[4] Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. IEEE Computer, 42(8), 30–37.

[5] Covington, P., Adams, J., & Sargin, E. (2016). Deep neural networks for YouTube recommendations. Proceedings of ACM RecSys.

[6] Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

[7] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. JMLR, 12, 2825–2830.

[8] The Movie Database (TMDB). (2023). TMDB API Documentation v3. https://developers.themoviedb.org/3

[9] Kaggle. (2017). TMDB 5000 Movie Dataset. https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata

[10] Streamlit Inc. (2023). Streamlit Documentation. https://docs.streamlit.io

[11] Porter, M. F. (1980). An algorithm for suffix stripping. Program, 14(3), 130–137.

[12] Burke, R. (2002). Hybrid recommender systems: Survey and experiments. UMUAI, 12(4), 331–370.

Appendix

A. Project Directory Structure

```
movie-recommendation-system/
├── data/  tmdb_5000_movies.csv  +  tmdb_5000_credits.csv
├── notebooks/  movie_recommender.ipynb
├── models/  movie_list.pkl  +  similarity.pkl
├── app.py                   (Streamlit application)
├── preprocessing.py         (Data cleaning & feature engineering)
├── requirements.txt         (Python dependencies)
└── README.md                (Project documentation)
```

B. Python Dependencies

```
pandas>=1.5.0      numpy>=1.23.0      scikit-learn>=1.1.0
nltk>=3.7          streamlit>=1.20.0  requests>=2.28.0
```